

AI Code Review & Security Analysis Report

Professional Code Quality & Security Assessment

Generated: 2026-08-14 10:07 UTC | Language: python | Model: llama-3.3-70b-versatile

Executive Summary

Code Health Score	Risk Score	Total Findings
0/100	33	14

Merge Readiness: **Not Ready**

The code review has identified 14 findings with a risk score of 33. The majority of the findings are High severity, indicating significant security and code quality concerns.

Severity Breakdown

Critical	High	Medium	Low
0	7	5	2

Detailed Findings

Type	Severity	Line	Message	Recommendation
1. Deep Nesting	● Medium	384	Function 'prompt_column' has nesting depth 5.	Flatten nested logic with guard clauses or helper functions.
2. Long Method	● Medium	469	Function '_add_customer_row' contains 21 statements.	Split the function into smaller units with single responsibilities.
3. Deep Nesting	● Medium	929	Function 'main' has nesting depth 16.	Flatten nested logic with guard clauses or helper functions.
4. High Cyclomatic Complexity	● High	1	Complexity = 126	Reduce nested decision points.
5. Hardcoded Password	● High	30	ORACLE_AMAN = dict(user="SYSTEM", password="password",	Store passwords in environment variables.
6. Hardcoded Password	● High	32	ORACLE_UMAPATHI = dict(user="SYSTEM", password="umapathi",	Store passwords in environment variables.

Type	Severity	Line	Message	Recommendation
7. Hardcoded Password	● High	35	user="root", password="tharun")	Store passwords in environment variables.
8. Hardcoded Password	● High	37	user="postgres", password="toor")	Store passwords in environment variables.
9. Hardcoded Password	● High	39	user="postgres", password="Vignesh")	Store passwords in environment variables.
10. Security	● High	23	Hardcoded database credentials	Use environment variables or a secure secrets manager to store database credentials
11. Error Handling	● Medium	144	Broad exception handling	Handle specific exceptions instead of catching the general Exception class
12. Code Quality	● Low	50	Magic strings	Define constants for magic strings to improve code readability
13. Security	● Medium	250	Potential SQL injection vulnerability	Use parameterized queries to prevent SQL injection attacks
14. Code Quality	● Low	300	Long function	Break down long functions into smaller, more manageable functions

■ Remediation Roadmap

Priority-based action items to improve code quality and security

Prior ity	Finding Type	Severity	Impact	Suggested Action
#1	Security	● High	Hardcoded database credentials	Use environment variables or a secure secrets manager to store database credentials
#2	Hardcoded Password	● High	Hardcoded passwords	Store passwords in environment variables
#3	High Cyclomatic Complexity	● High	Complexity = 126	Reduce nested decision points
#4	Security	● Medium	Potential SQL injection vulnerability	Use parameterized queries to prevent SQL injection attacks
#5	Error Handling	● Medium	Broad exception handling	Handle specific exceptions instead of catching the general Exception class
#6	Deep Nesting	● Medium	Function 'prompt_column' has nesting depth 5	Flatten nested logic with guard clauses or helper functions
#7	Long Method	● Medium	Function '_add_customer_row' contains 21 statements	Split the function into smaller units with single responsibilities
#8	Deep Nesting	● Medium	Function 'main' has nesting depth 16	Flatten nested logic with guard clauses or helper functions
#9	Code Quality	● Low	Magic strings	Define constants for magic strings to improve code readability
#10	Code Quality	● Low	Long function	Break down long functions into smaller, more manageable functions

■ Key Recommendations

1. Address all Critical and High severity findings before merging.
2. Run additional security scanning tools to complement this analysis.
3. Consider implementing automated testing to catch similar issues in the future.

■ Code Quality Metrics

Overall Health Score: 0/100

Risk Level: Unknown

Maintainability: N/A

Security Score: N/A

■ Source Code Snapshot

```
"""
Experiment 6.3 -- Banking Distributed Database, 5 nodes -- Interactive Tool
-----

Sites:
oracle_aman : Oracle -- region NORTH
oracle_uma : Oracle -- region SOUTH
mysql_tharun : MySQL -- region EAST
pg_shiva : Postgres -- region WEST
pg_vignesh : Postgres -- region CENTRAL

Install once:
pip install oracledb psycpg2-binary mysql-connector-python tabulate

Run:
python federation.py

IMPORTANT: the CONFIG values below are PLACEHOLDERS. Replace user/password/
dsn/host/dbname with whatever you ACTUALLY used when you ran your own
site_*.sql script -- e.g. if you connected to Oracle as "system" or as
some other existing schema, put THAT username/password here, not a made
up one. A wrong username/password here is not a bug in this script -- it
just means the credentials don't match a real login yet.
"""

import sys
import traceback
from tabulate import tabulate

# ===== CONFIG =====
ORACLE_AMAN = dict(user="SYSTEM", password="password",
dsn="100.109.6.43:1521/XEPDB1")
ORACLE_UMAPATHI = dict(user="SYSTEM", password="umapathi",
dsn="100.64.73.94:1521/FREEPDB1")
MYSQL_THARUN = dict(host="100.81.28.11", port=3306, database="banking_east",
user="root", password="tharun")
POSTGRES_SHIVA = dict(host="100.77.94.73", port=5432, dbname="Banking_West",
user="postgres", password="toor")
POSTGRES_VIGNESH = dict(host="100.120.29.0", port=5432, dbname="chinook",
user="postgres", password="Vignesh")
# =====

SITES = {
"1": dict(key="oracle_aman", label="Oracle_Aman (NORTH)", engine="oracle", cfg=ORACLE_AMAN),
"2": dict(key="oracle_uma", label="Oracle_Umapathi (SOUTH)", engine="oracle", cfg=ORACLE_UMAPATHI),
"3": dict(key="mysql_tharun", label="MySQL_Tharun (EAST)", engine="mysql", cfg=MYSQL_THARUN),
"4": dict(key="pg_shiva", label="Postgres_Shiva (WEST)", engine="postgres", cfg=POSTGRES_SHIVA),
"5": dict(key="pg_vignesh", label="Postgres_Vignesh (CENTRAL)", engine="postgres",
cfg=POSTGRES_VIGNESH),
}
```

```
KYC_GROUPS = {  
  "oracle_aman": ("kyc_group_a", ["customer_id", "date_of_birth", "gender"]),  
  "oracle_uma": ("kyc_group_b", ["customer_id", "national_id", "pan_number"]),  
  "mysql_tharun": ("kyc_group_c", ["customer_id", "email", "phone"]),  
  "pg_shiva": ("kyc_group_d", ["customer_id", "address_line", "city"]),  
  "pg_vignesh": ("kyc_group_e", ["customer_id", "occupation", "annual_income"]),  
}
```

```
# The 4 horizontal fragment tables exist, identically named, at every site.  
HORIZONTAL_TABLES = ["customers", "accounts", "transactions", "loan_accounts"]
```

```
# LOAN_PRODUCTS, CURRENCY_EXCHANGE_RATES and FEE_SCHEDULE are all FULLY  
# REPLICATED (Experiment 7): identical rows, identical columns, present
```

```
... [Code truncated for brevity] ...
```



This report was generated by the AI Code Review & Security Analysis Agent. Review all findings carefully before implementing changes.
Report generated on 2026-08-14 10:07 UTC